# Studio Nx

Backend(FastAPI/Python) 20 — check_nickname()

2026-10-12

# Backend(FastAPI/Python) 20 — `check_nickname()`

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/services/auth_service.py` |
| 작업 | P1-04 회원가입: 닉네임 중복 확인 (PRD 01 FR-101, FR-108) |
| 작성 | Claude (AI), 2026-10-07, 사용자 검토 |
| 이전 · 다음 | 19 `check_email()` · 21 `signup()` |

## 1. 하는 일

이 닉네임을 **새로 쓸 수 있는지** 알려 줍니다. 판단 규칙은 19번과 같습니다 (`availability_of()`를 함께 씁니다).

| DB에서 그 닉네임을 가진 행 | 결과 |
|---|---|
| 없음 | `{"available": true}` |
| 활동 회원, 정지 회원 | `{"available": false, "reason": "TAKEN"}` |
| 탈퇴한 지 3일이 안 됨 | `{"available": false, "reason": "REJOIN_TOO_SOON", "available_at": "…Z"}` |
| 탈퇴한 지 3일이 지남 | `{"available": true}` (가입할 때 18번으로 비켜 둡니다) |

### 대소문자가 달라도 같은 닉네임으로 봅니다

| 이미 있는 닉네임 | 확인하는 닉네임 | 결과 |
|---|---|---|
| `beatmaker` | `beatmaker` | `TAKEN` |
| `beatmaker` | `BeatMaker`, `BEATMAKER` | `TAKEN` |
| `beatmaker` | `" beatmaker "` (앞뒤 공백) | `TAKEN` |
| `café` | `cafe` | `TAKEN` (악센트도 구분하지 않습니다) |

## 2. 언제 쓰나

| 부르는 곳 | `release` | 화면 |
|---|---|---|
| `GET /auth/check-nickname` (24) | `False` | join.html 닉네임 칸의 **Check** 버튼 |
| 21 `signup()` | `True` | **Sign Up** 버튼 |
| P1-08 `update_me()` | `False` 또는 `True` | userInfo.html의 닉네임 바꾸기 (그때 정합니다) |

## 3. 코드

**import** (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.orm import Session
```

```
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import입니다 (19번과 같고, 이번에 새로 추가한 것은 없습니다). 이 함수는 **`select`, `Session`, `User`, `AvailabilityResponse`**를 쓰고, 판단은 19번의 `availability_of()`에 맡깁니다.

**함수**

```
def check_nickname(db: Session, nickname: str, release: bool = False) -> AvailabilityResponse:
    user = db.scalar(select(User).where(User.nickname == nickname.strip()))
    return availability_of(db, user, release)
```

## 문법 풀이

| import 문법 | 뜻 |
|---|---|
| `from 모듈 import 이름` | 모듈 안의 `이름`만 가져와서 그대로 씁니다 |

| 식 | 뜻 |
|---|---|
| `nickname.strip()` | 앞뒤 공백만 지웁니다. 이메일과 달리 **대소문자는 바꾸지 않습니다** |
| `User.nickname == …` | SQL `WHERE nickname = …`입니다. 실제 비교는 **MySQL이 하며**, 아래의 정렬 규칙을 따릅니다 |

## MySQL의 비교 규칙 (정렬 규칙 `utf8mb4_0900_ai_ci`)

`users.nickname`과 `users.email` 칸은 이 규칙을 씁니다. 아래는 이 DB에서 직접 확인한 결과입니다.

| 비교 | 결과 | 뜻 |
|---|---|---|
| `'Beat' = 'beat'` | 같음 | `ci`는 case-insensitive, 즉 대소문자를 구분하지 않는다는 뜻입니다 |
| `'café' = 'cafe'` | 같음 | `ai`는 accent-insensitive, 즉 악센트를 구분하지 않는다는 뜻입니다 |
| `'ａ' = 'a'` | 같음 | 전각 글자도 같게 봅니다 |
| `'비트' = '비트 '` | **다름** | 뒤 공백을 그대로 비교합니다 (`NO PAD`). 그래서 `strip()`이 필요합니다 |

# 4. 흐름

1. 앞뒤 공백을 지웁니다.
2. 그 닉네임을 가진 행을 DB에서 찾습니다 (대소문자와 악센트는 무시됩니다).
3. 19번의 `availability_of()`로 판단합니다.

# 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 닉네임은 소문자로 바꾸지 않음 | 닉네임은 화면에 보이는 이름입니다. `BeatMaker`로 가입한 사람의 표기를 그대로 지켜 줍니다 |
| 그래도 `BeatMaker`와 `beatmaker`는 같은 이름으로 봄 | DB의 `UNIQUE`가 이미 그렇게 비교합니다. 다르게 확인하면 "✓ Available"이라고 해 놓고 가입할 때 DB가 거절하는 일이 생깁니다. 또 `beatmaker`를 흉내 낸 `BeatMaker` 계정(사칭)을 막는 효과도 있습니다 |

| 선택 | 이유 |
|---|---|
| 비교를 파이썬이 아니라 DB에 맡김 | 확인 결과와 가입할 때의 DB 판단이 항상 같아집니다 |
| 앞뒤 공백은 직접 지움 | MySQL은 뒤 공백을 다르게 봅니다(`NO PAD`). `"beatmaker "`로 따로 가입되면, 화면에서는 똑같아 보이는 이름이 둘 생깁니다 |
| 판단은 19번과 같은 함수로 | 탈퇴 3일 규칙과 정지 회원 처리가 이메일과 똑같아야 합니다 |

## 6. 시험 (`tests/test_auth_service.py`에 8개를 추가해서 모두 27개, 실제 MySQL로 실행하고 rollback)

| 시험 | 확인한 것 |
|---|---|
| 없는 닉네임 | `available=True` |
| 같은 닉네임 | `TAKEN` |
| 공백과 대소문자 (3개) | `" beatmaker "`, `BeatMaker`, `BEATMAKER`가 모두 `TAKEN`입니다 |
| DB도 같은 규칙인지 | `beatmaker`가 있을 때 `BeatMaker`로 넣으면 DB가 거절합니다 (`IntegrityError`). 확인 결과와 일치합니다 |
| 탈퇴한 지 1일 | `REJOIN_TOO_SOON` |
| 탈퇴한 지 5일 | 확인만 하면 그대로 두고, `release=True`면 `deleted_user_…`로 비켜 둔 뒤 같은 닉네임으로 가입됩니다 |

전체 시험 135개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.